Delegados

Objetivo:

- Conocer los conceptos relacionados con delegados para enviar métodos como parámetros

- Desarrollar aplicaciones con delegados.

Ingeniería y Sistemas
UFG UNIVERSIDAD FRANCISCO GAVIDIA
Powered by Arizona State University®

La plataforma .NET utiliza una serie de tipos de datos que son compartidos por todos sus lenguajes (incluyendo C#) independientemente del compilador utilizado, ya que en realidad el tipo de dato no está definido en el lenguaje, sino en el sistema de datos comunes.

Un delegado es un nuevo tipo que, a diferencia de los tipos comunes, no manipula un dato, sino que hace referencia a un método.

Esto significa que el delegado tiene la dirección de memoria de un método a través de la cual lo Accesa, así que para invocar un método es necesario hacerlo por medio de un apuntador debidamente controlado.

Esto trae como consecuencia que si un delegado es un tipo, entonces se puedan declarar variables de él; sin embargo, estas variables no almacenan datos, sino referencias a métodos.

Los Delegados son tipos especiales que se utilizan para encapsular un método dentro de una firma específica.

Los delegados son objetos especiales que pueden contener una referencia a un método con un firma específica.

Un delegado se define utilizando la palabra reservada delegate.

se puede definir un delegado con la siguiente Sintaxis:

Sintaxis delegado

<Modificador de acceso> delegate <tipo de retorno> <nombre>(<parametros>[])

Ingeniería y Sistemas
UFG UNIVERSIDAD FRANCISCO GAVIDIA
Powered by Arizona State University®

Se puede definir un delegado como se indica a continuación:

```
// Declare a delegate
public delegate void RectangleHandler(Rectangle rect);
delegate void Del(int i, double j);
delegate void Del();
delegate int OperarDelegate(int a, int b);
delegate ImprimirDelegate(string msj);
```

Ejemplo en consola

```
// Declarando un delegate
public delegate void Imprimir(string mensaje);
2 referencias
class Program
{
    //metodo imprimir por pantalla
    1 referencia
    private void ImprimirPantalla(string msj)
    {
        Console.WriteLine(msj);
    }
    0 referencias
    static void Main(string[] args)
    {
        Program p = new Program();
        //instancia del delegado y pasando el metodo como parametro
        Imprimir imprimirDelegado = new Imprimir(p.ImprimirPantalla);
        //invocando el metodo a travéz del delegado
        imprimirDelegado("Hola Mundo con delegados");
        Console.ReadKey();
    }
}
```

delegado que reciba un string y no retorna.

- El método que pasa como un parámetro de delegado debe tener la misma firma que la declaración de delegado.
- Es importante tener en cuenta que el tipo de retorno y los parámetros de entrada al método deben coincidir entre método y delegado.
- Una instancia de delegado puede encapsular un método de instancia o estático
- Los tipos de delegado se derivan de la clase Delegate en .NET. Los tipos de delegados son sealed (no se pueden derivar) y no se pueden derivar clases personalizadas de Delegate

Un delegado es un tipo que representa referencias a métodos con una lista de parámetros determinada y un tipo de valor devuelto.

Cuando se crea una instancia de un delegado, puede asociar su instancia a cualquier método mediante una signatura compatible y un tipo de valor devuelto.

Ejemplo en consola

delegado que reciba dos enteros y retorne un entero.

```
// Declarando un delegate
delegate int Operar(int x1, int x2);
2 referencias
class Program
{
    //metodo sumar
    2 referencias
    public int Sumar(int x, int y)
    {
        return x + y;
    }
    //metodo restar
    2 referencias
    public int Restar(int x, int y)
    {
        return x - y;
    }
    0 referencias
    static void Main(string[] args)
    {
        Program p = new Program();
        Console.WriteLine("Suma y resta llamando directamente a los métodos.");
        Console.WriteLine(p.Sumar(2, 2));
        Console.WriteLine(p.Restar(2, 2));
        Operar miDelegado = p.Sumar;
        Console.WriteLine("Suma y resta llamando a los métodos a través de delegados");
        Console.WriteLine(miDelegado(2, 2));
        miDelegado = p.Restar;
        Console.WriteLine(miDelegado(2, 2));
        Console.ReadKey();
    }
}
```

¿Para qué sirven los delegados?

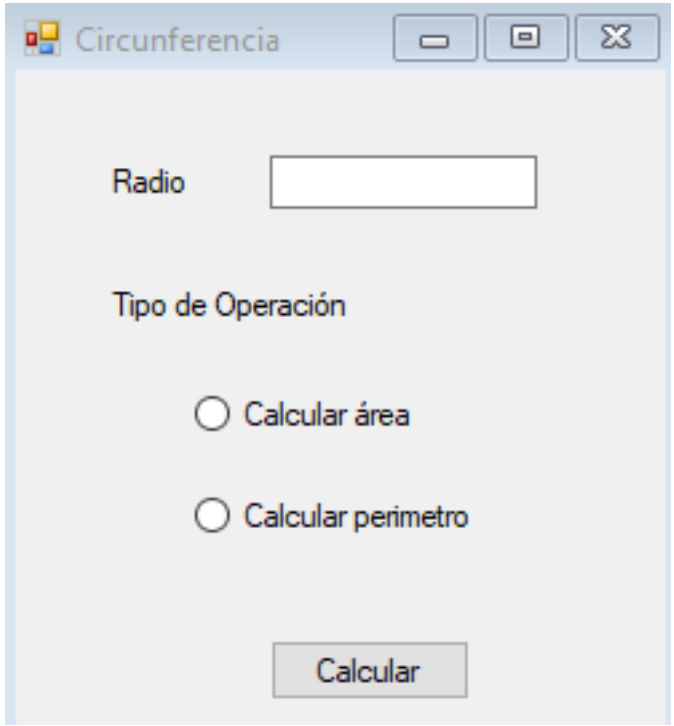
- Los delegados se utilizan para pasar (enviar) métodos como parámetros a otros métodos.

- De esta forma, un método puede recibir un delegado como parámetro e invocarlo.

- Al diseñar un delegado para encapsular un método de instancia, el delegado hace referencia tanto a la instancia como al método y puede hacer referencia a cualquier tipo de objeto siempre que haya un método en ese objeto que coincida con la firma del delegado.

- Al diseñar un delegado para encapsular un método estático, sólo hace referencia al método

Calcular área y perímetro de Circunferencia



Para la aplicación visual se diseña un formulario que contiene

- Un TextBox (txtRadio) para capturar el valor del radio de la circunferencia
- Dos Radio Buttons (radArea y radPerimetro) para seleccionar la operación deseada
- Un botón (btnCalcular) que ejecuta el cálculo indicado
- Se muestra el resultado mediante un MessageBox

Ingeniería y Sistemas
UFG UNIVERSIDAD FRANCISCO GAVIDIA
Powered by Arizona State University®

Diagrama de la Caja Negra planteamiento del problema

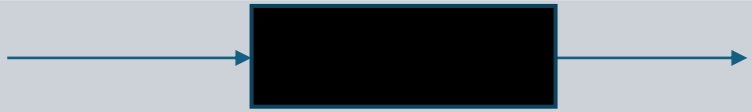
Entrada	Proceso / Solución	Salida
La longitud del radio de una circunferencias		Impresión de área y Perímetro del circulo
Radio	Clase Circunferencia $A = \pi \ radio^{2}$ $P = 2 \pi r$	Área Perímetro

Diagrama de clase con el delegado que calcula área y perímetro de una circunferencia

Circunferencia
- dblRadio: double
+ radio {get; set;} : double + CalcularArea() : double + CalcularPerimetro : double + Calcular(miOperacion: Operacion) : double

Método que recibe como parámetro el nombre del método que se desea invocar (del tipo delegado Operacion)

Ingeniería y Sistemas
UFG UNIVERSIDAD FRANCISCO GAVIDIA
Powered by Arizona State University®

Creamos la nueva clase para el programa que calcula el Área y el Perímetro

```
public class Circunferencia
{
    //Atributos privados
    private double dblRadio;

    //propiedad
    3 referencias
    public double Radio
    {
        get { return dblRadio; }
        set { dblRadio = value; }
    }

    //Método para calcular el área
    1 referencia
    public double CalcularArea()
    {
        return Math.PI * Math.Pow(Radio, 2);
    }

    //Método para calcular el Perímetro
    1 referencia
    public double CalcularPerimetro()
    {
        return Math.PI * Radio * 2;
    }
```

Creamos el delegado que llamara a los métodos

```
    //Delegado público
    public delegate double Operacion();
    //Método que recibe como parámetro el método que desea invocar (del tipo del delegado)
    2 referencias
    public double Calcular(Operacion miOperacion)
    {
        double Resultado = miOperacion();
        return Resultado;
    }
}
```

Código fuente del programa

```
1 referencia
private void btnCalcular_Click(object sender, EventArgs e)
{
    //Declaración y creación de un objeto local
    Circunferencia miCircunferencia = new Circunferencia();
    try
    {
        //intenta capturar el radio de la circunferencia
        miCircunferencia.Radio = Convert.ToDouble(txtRadio.Text);
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message);
        txtRadio.Text = string.Empty;
        txtRadio.Focus();
        return;
    }

    double resultado = 0;
    if (radArea.Checked)
    {
        resultado = miCircunferencia.Calcular(miCircunferencia.CalcularArea);
        MessageBox.Show("Área = " + resultado);
    }
    else
    {
        resultado = miCircunferencia.Calcular(miCircunferencia.CalcularPerimetro);
        MessageBox.Show("Perímetro = " + resultado);
    }
}
```

Revisión de código fuente del programa

```
//Declaración y creación de un objeto local
Circunferencia miCircunferencia = new Circunferencia();
```

Al oprimir el botón para calcular (btnCalcular) se crea un objeto local (miCircunferencia)

```
//intenta capturar el radio de la circunferencia
miCircunferencia.Radio = Convert.ToDouble(txtRadio.Text);
```

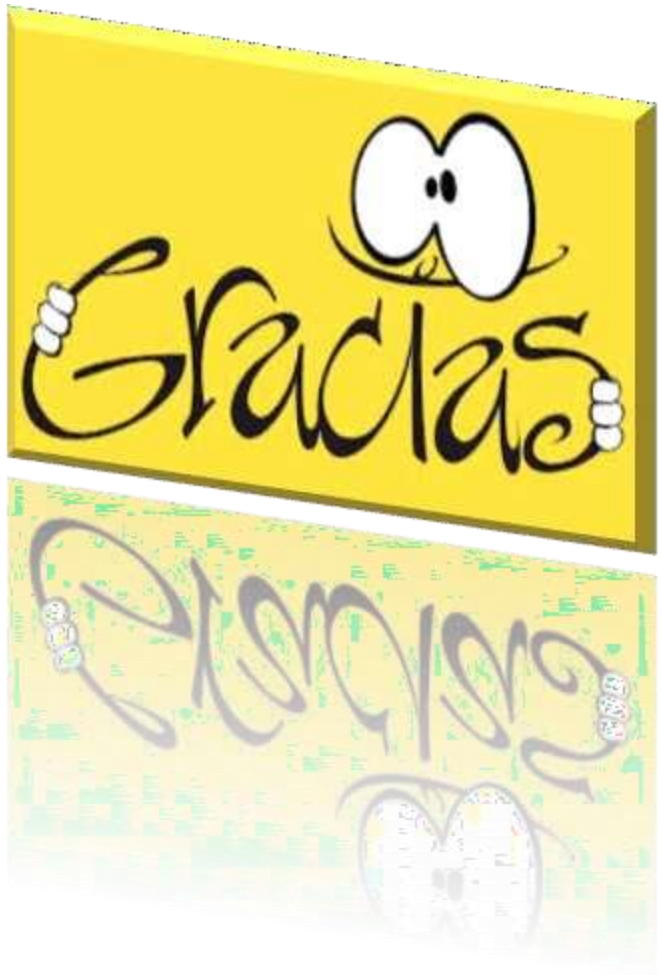
Se le inserta el valor del radio capturado en txtRadio y se invoca su método Calcular()

```
if (radArea.Checked)
{
    resultado = miCircunferencia.Calcular(miCircunferencia.CalcularArea);
    MessageBox.Show("Área = " + resultado);
}
else
{
    resultado = miCircunferencia.Calcular(miCircunferencia.CalcularPerimetro);
    MessageBox.Show("Perímetro = " + resultado);
}
```

Se envía como parámetro el nombre del método con la operación indicada en el radioButton seleccionado.

Si se selecciona el radioButton para calcular el área (radArea), entonces se invoca miCircunferencia.Calcular(miCircunferencia.CalcularArea).

Si el usuario escoge el radioButton para calcular el perímetro (radPerimetro), invoca miCircunferencia.Calcular(miCircunferencia.CalcularPerimetro)



Ingeniería y Sistemas

UFG
UNIVERSIDAD FRANCISCO GAVIDIA
Powered by Arizona State University®